A Canonical Unary Rewrite Calculus for Balanced Ternary and a Locality Obstruction for Addition

Status: publication draft — **READY FOR EXTERNAL REVIEW**.

Abstract

Balanced ternary represents integers with digits in $\{-1, 0, +1\}$. Its natural digit operators are

$$I_a(x) = a + 3x, \qquad D(n) = \frac{n - \mathrm{lsd}(n)}{3}, \qquad S(n) = 3n, \qquad N(n) = -n.$$

We define a one-hole unary term language generated by $\{D, I_a, S, N\}$ and give it a finite rewrite system. The system terminates and is confluent; every term has a unique syntactic normal form, rewriting preserves integer-function semantics, and distinct irreducible terms denote distinct maps $\mathbb{Z} \to \mathbb{Z}$.

We then isolate the exact boundary at addition. Call a binary output H D-local if

$$H(x, y) = G(D(x), D(y))$$

for some $G : \mathbb{Z}^2 \to \mathbb{Z}$. We prove that the next-state output $H(x, y) = D(x + y)$ is not D-local. The minimal witness is $D(0) = D(1) = 0$, while $D(0 + 0) = 0$ and $D(1 + 1) = 1$. The exact identity

$$D(x + y) = D(x) + D(y) + \mathrm{carry}(\mathrm{lsd}(x), \mathrm{lsd}(y))$$

identifies the discarded information. Finally, the natural carry-free extension that pushes S through addition is not locally confluent at $D \circ S$. These concrete results are verified in Lean. They do not classify arbitrary rewrite systems for arithmetic.

1. Introduction

Every integer has a unique balanced-ternary expansion

$$n = \sum_{i \geq 0} a_i 3^i, \qquad a_i \in \{-1, 0, +1\}.$$

Unique expansion is classical. So are signed-digit addition algorithms and the general methods used to prove termination and confluence. The question here is narrower:

> Do the natural balanced-ternary unary operators form a canonical rewrite calculus, and what exact obstruction appears when addition is required to use the same local state?

The answer has four parts.

1. The unary language generated by $\{D, I_a, S, N\}$ has a complete canonical presentation on its specified one-hole syntax.
2. The output $D(x + y)$ does not factor through $(D(x), D(y))$.
3. The balanced carry identity identifies the missing least-significant-trit information.
4. The natural carry-free push-in extension has a non-joining critical peak at $D \circ S$.

The first result is a concrete application of standard rewriting theory. The second is the central locality theorem. The third explains its arithmetic content, and the fourth shows how the same obstruction appears syntactically.

No claim is made that addition lacks a finite presentation. Coefficient-word normalization is an explicit complete treatment of integer sums, but it retains normalization state absent from the unary tree calculus.

2. Balanced-ternary unary language

Let $\mathrm{lsd}(n) \in \{-1, 0, +1\}$ be the balanced residue of n modulo 3. Define

$$D(n) = \frac{n - \mathrm{lsd}(n)}{3}, \qquad I_a(n) = a + 3n, \qquad S(n) = 3n, \qquad N(n) = -n.$$

Here $a \in \{-1, 0, +1\}$, and $I_0 = S$ as integer maps.

The formal term grammar is

```
t ::= var | D(t) | I-(t) | I0(t) | I+(t) | S(t) | N(t).
```

It is a one-hole unary grammar. It contains neither `Add` nor `Mul`, and it is not the operator-word string language used elsewhere in the project.

Evaluation assigns an integer function to each term:

$$[\![-]\!] : \mathrm{OpFrag} \to (\mathbb{Z} \to \mathbb{Z}).$$

For example,

$$[\![D(t)]\!](n) = D([\![t]\!](n)), \qquad [\![I_a(t)]\!](n) = I_a([\![t]\!](n)).$$

Semantic equality below means equality as functions on every integer, not agreement at one input.

3. Canonical rewrite calculus

The rewrite relation is generated by the following root rules and closed under every unary context:

```
I0(t)      → S(t)
D(I-(t))   → t
D(I0(t))   → t
D(I+(t))   → t
D(S(t))    → t
N(N(t))    → t
N(S(t))    → S(N(t))
N(I0(t))   → S(N(t))
N(I-(t))   → I+(N(t))
N(I+(t))   → I-(N(t))
N(D(t))    → D(N(t)).
```

Every rule is an exact identity of integer functions.

The final orientation is necessary for semantic canonicity. Without $N(D(t)) \to D(N(t))$, the terms $N(D(\mathrm{var}))$ and $D(N(\mathrm{var}))$ are distinct irreducibles with identical integer semantics.

3.1 Canonical-form theorem

Theorem 1 (canonical unary calculus). On `OpFrag`:

1. every rewrite sequence terminates;

2. the relation is locally confluent and hence confluent;
3. every term has a unique syntactic normal form;
4. rewriting preserves evaluation on $\mathbb{Z}$;
5. two irreducible terms with equal evaluation on every integer are syntactically equal.

Consequently, every term has a unique normal descendant, and that descendant is the unique irreducible representative of its integer function within this grammar.

The Lean façade theorem is `unary_complete_canonical_form`.

3.2 Proof structure

Termination uses the lexicographic rank

$$(\#I_0,\ N\text{-inversions},\ \text{term size}).$$

An N-inversion is a pair consisting of an N-node and a pushable descendant among D, I_-, I_0, I_+, S. Cancellation decreases size, $I_0 \to S$ decreases the first coordinate, and moving N inward decreases the second.

The finite family of root and root-versus-inner critical pairs joins. Termination and local confluence give confluence by Newman's lemma. Semantic injectivity of irreducibles then follows from the normal-form grammar and uniqueness of balanced-ternary words.

The general proof method is standard. The project-specific object is the exact operator fragment, including the necessary orientation of N past D.

4. D-locality and the main theorem

4.1 Definition

A binary integer output $H : \mathbb{Z}^2 \to \mathbb{Z}$ is D-local if there exists $G : \mathbb{Z}^2 \to \mathbb{Z}$ such that

$$\forall x, y \in \mathbb{Z}, \qquad H(x, y) = G(D(x), D(y)).$$

Thus H is D-local exactly when it factors through the pair of operand D-states. This is the only locality notion used here.

For addition, the relevant output is not $x + y$ itself but its next state

$$H_{\mathrm{add}}(x, y) = D(x + y).$$

4.2 Exact non-locality

Theorem 2 (addition next-state non-locality).

$$\boxed{\neg\exists G : \mathbb{Z}^2 \to \mathbb{Z} \quad \forall x, y, \quad D(x + y) = G(D(x), D(y)).}$$

Proof. The inputs 0 and 1 have the same D-state:

$$D(0) = D(1) = 0.$$

Therefore the pairs $(0,0)$ and $(1,1)$ present the same argument $(0,0)$ to any proposed G. But

$$D(0+0) = 0, \qquad D(1+1) = D(2) = 1.$$

No such G exists. □

The Lean theorem is `add_not_DLocal`. Its predicate `DLocal` is the factorization definition above, applied to `fun x y => DZ (x + y)`.

The result is stronger than the failure of the single formula $D(x+y) = D(x) + D(y)$: it rules out every function of $(D(x), D(y))$ alone.

5. Addition and the missing carry

Balanced-ternary addition satisfies the exact identity

$$\boxed{D(x+y) = D(x) + D(y) + \mathrm{carry}(\mathrm{lsd}(x), \mathrm{lsd}(y)).}$$

In Lean, the carry is the second component of `addDigit (lsdZ x) (lsdZ y)`. It belongs to $\{-1, 0, +1\}$.

The identity explains Theorem 2. The states $D(x)$ and $D(y)$ retain the higher parts of the operands after their least-significant trits have been removed. Those removed trits determine whether their sum leaves the digit set and produces a carry into the next position. Hence the carry is not an implementation accident: it is precisely the information absent from $(D(x), D(y))$.

At the minimal witness,

$$D(1+1) = D(2) = 1 \neq 0 = D(1) + D(1).$$

The Lean theorem `carry_of_add` is a rearrangement of the existing exact theorem `D_add`.

6. Natural carry-free extension and critical peak

Consider the syntax

```
t ::= X | Y | D(t) | S(t) | Add(t,t)
```

with the named relation generated by

```
D(S(t))          → t
S(Add(t,u))      → Add(S(t),S(u))
```

and congruence. There is deliberately no rule pushing D through `Add`, because the carry-free candidate is unsound.

The term

$$D(S(\mathrm{Add}(X, Y)))$$

has two one-step descendants:

$$\mathrm{Add}(X, Y)$$

by $D(S(t)) \to t$, and

$$D(\mathrm{Add}(S(X), S(Y)))$$

by pushing S through `Add` inside D. Both descendants are irreducible in this named system, and they are syntactically distinct. Nevertheless, both evaluate to $x + y$.

Theorem 3 (named push-in obstruction). The named carry-free push-in relation is not locally confluent.

The Lean theorems are `pushIn_not_locally_confluent` and `pushIn_peak_semantic`.

This is a concrete obstruction to one natural extension. It is not a classification of every rewrite system containing addition, and it is not used to prove Theorem 2. Rather, it shows the same locality boundary appearing as a critical peak.

7. Restricted maximality question

Theorem 2 immediately excludes any exact first-digit implementation whose output $D(x + y)$ is required to be a function only of $(D(x), D(y))$. That is the precise locality boundary.

A stronger theorem about a class of complete tree rewrite systems would additionally require an independent syntactic definition of "carry-free" and a proof that completeness in that class forces D-locality. Defining the class by assuming D-locality, or by assuming that the two descendants in §6 remain irreducible, would only restate the conclusion. No such universal maximality statement is claimed here.

The broader project-level observation that the examined Add systems either remain incomplete or introduce constants, carry state, or coefficient normalization is retained as a human research record, not as a theorem of this note.

8. Related work

The novelty boundary has three parts.

1. **Balanced ternary and unique expansion.** Unique balanced-ternary expansion supplies the digit representation. It is not the operator calculus studied here.
2. **Signed-digit arithmetic.** Avizienis-style signed-digit arithmetic already provides carry-managed addition. The present result instead concerns a canonical unary operator language and the factorization boundary of its D-states.
3. **Rewriting theory.** Newman, Knuth–Bendix, and critical-pair methods provide the general termination and confluence framework. The contribution is their exact application to $\{D, I_a, S, N\}$, together with the formally verified locality obstruction at addition.

No new general rewriting theory, numeral system, or signed-digit addition algorithm is claimed.

9. Scope and limitations

The exact formal scope is:

- one-hole unary terms generated by $\{D, I_a, S, N\}$;
- integer-function semantics on $\mathbb{Z}$;
- output factorization through the pair $(D(x), D(y))$;
- the finite affine constructor class in Appendix A;
- the named push-in relation in §6.

The note does not claim:

- that addition has no finite rewrite presentation;
- that arbitrary finite Add-tree systems are incomplete;
- that every extension with additional state is a computer-algebra system;
- that the production operator-word table is confluent;
- that AC completion is impossible or unnecessary.

Coefficient-word normalization gives a complete representation of integer sums after evaluation. It is a different canonicalizer from the unary `OpFrag` tree system and is not imported into the locality proof.

10. Formal verification

The development uses Lean 4 and Mathlib. Its façade files are:

- `formal/BTCalculus/RewriteCore.lean`;
- `formal/BTCalculus/RewriteAddBoundary.lean`.

The principal theorem names are:

- `unary_complete_canonical_form`;
- `add_not_DLocal`;
- `carry_of_add`;
- `pushIn_not_locally_confluent`;
- `pushIn_peak_semantic`.

Supporting constructor results are `exactTriple_characterization`, `not_exact_Ip_Ip`, and `not_exact_Im_Im`. Lean also packages the boundary results as the secondary conjunction `add_requires_carry_state`.

The underlying unary proofs are in `OpFrag.lean`, `OpFragNewman.lean`, and `OpFragSemantic.lean`; the carry identity `D_add` is in `Algebra.lean`.

Verification command:

```
lake build BTCalculus
```

The formal files contain no `sorry`, `admit`, or added axioms. Lean verifies the stated concrete theorems; it is not itself the novelty claim.

11. Conclusion

The one-hole unary language generated by $\{D, I_a, S, N\}$ has a finite canonical rewrite presentation under exact integer-function semantics. Its normal forms are unique both syntactically and as representatives of the functions expressible in the grammar.

The next-state output of exact addition lies beyond the same local state model:

$$D(x + y) \neq G(D(x), D(y))$$

for every possible G. The carry identity identifies the missing least-significant-trit information, and the named push-in system shows the same boundary as a non-confluent critical peak. This is a precise locality obstruction, not an impossibility theorem for arithmetic.

Appendix A. Constructor-sum classification

Let

$$\mathcal{A} = \{S, I_+, I_-, N\}.$$

Write each constructor as $U(t) = s_U t + c_U$, where

$$(s_U, c_U) \in \{(3,0), (3,1), (3,-1), (-1,0)\}.$$

Then

$$\forall x, y, \quad U(x) + V(y) = W(x+y)$$

holds exactly when

$$s_U = s_W, \qquad s_V = s_W, \qquad c_U + c_V = c_W.$$

The Lean theorem `exactTriple_characterization` gives the eight concrete triples

```
(S,S,S)        (N,N,N)
(I+,S,I+)      (S,I+,I+)
(I-,S,I-)      (S,I-,I-)
(I+,I-,S)      (I-,I+,S).
```

These are six parameterized rows when the two signs are written with I_a.

In particular,

$$I_+(x) + I_+(y) = S(x+y) + 2, \qquad I_-(x) + I_-(y) = S(x+y) - 2.$$

The constants ± 2 are not balanced trits, so neither same-sign sum is represented by a constructor in $\mathcal{A}$.

Appendix B. Closed exploration record

The discovery phase also studied operator-word fragments. The full production word table is not locally confluent, while three named opt-in fragments have separate human Newman certificates. Those results are archived in `docs/theory/rewrite_calculus.md` and are not part of the theorem package here. No word-table enlargement is proposed.

References and source map

- M. H. A. Newman, *On theories with a combinatorial definition of equivalence* (1942), registry id `newman-1942-confluence`.
- F. Baader and T. Nipkow, *Term Rewriting and All That* (1998), registry id `baader-nipkow-1998-term-rewri`
- A. Avizienis, *Signed-Digit Number Representations for Fast Parallel Arithmetic* (1961), registry id `avizienis-1961-signed-digit`.
- D. E. Knuth, *The Art of Computer Programming, Volume 2*, registry id `knuth-taocp-vol2`.

Canonical project records:

- `docs/problems/rewrite_calculus.md`;
- `docs/theory/rewrite_calculus_formalization.md`;
- `docs/theory/rewrite_calculus_reviewer_packet.md`;
- ledger rows `BTC-op-fragment-nd-nf`, `BTC-op-fragment-nd-semantic`, `BTC-add-not-D-local`, and `BTC-push-in-S-peak`.